# Software Requirements Specification

**for**

# Escape Sequence

**Version 2.0 approved**

**Prepared by: Akera Griffith & Kaitlyn Morris**

**CS 491**

**October 20th, 2025**

# Table of Contents



# Revision History

| Name | Date | Reason For Changes | Version |
|---|---|---|---|
| Kaitlyn & Akera | 10/12/25 | Adding Interface Screenshots | 2.0 |

# 1. Introduction

## 1.1 Purpose

This document, Software Requirements Specification (SRS), presents the detailed functional and nonfunctional requirements for the Escape Sequence game.

This is a product of The Space Cadets team (Kaitlyn Morris & Akera Griffith) for the CS 491 Software Engineering Project class supervised by Dr. Zhao. The game is a Java-based interactive blackjack variation done as a 2D first-person game. This document serves as an outline of software requirements for Escape Sequence version 1.0, including user interactions, functional behavior, external interfaces, and constraints.

The SRS describes the software system in detail, such as the single-player and multiplayer modes, the game graphical user interface (GUI), database components for user data and scores, and the rules for game logic. There is no detailed description of the hardware or external subsystems except that which is needed to support the software.

## 1.2 Document Conventions

This document follows the IEEE 830-1998 standard for Software Requirements Specifications.

- **Section headings** follow a hierarchical numbering scheme (1.0, 1.1, 1.2) for clarity.
- **Bold text** indicates section titles and key terms.
- *Italic text* highlights variable names, file names, and in-game elements.
- Requirements are categorized by priority where applicable:
  - **(H)** – High Priority: essential features for core functionality.
  - **(M)** – Medium Priority: important but not essential for initial release.
  - **(L)** – Low Priority: optional or future enhancements.
- Diagrams (use case and sequence diagrams) are labeled according to their section reference (Figure 3.1).

## 1.3 Intended Audience and Reading Suggestions

This SRS targets readers from different backgrounds and objectives. These include:

- Developers: To refer to when coding the game's logic, GUI, and multiplayer system.
- Project Manager (Dr. Zhao): To keep track of the development and ensure that the design is aligned with project objectives.
- Testers: To draft test cases and verify that the implementation of each requirement is accurate.
- Users (Players): To get acquainted with the features, the rules, and the functioning of the interface of the gameplay.

- Documentation Writers: To prepare help guides or user manuals by observing the system behavior stated.

The readers should first familiarize themselves with Section 1 (Introduction) which gives a summary and then with Section 2 (Overall Description) that describes the game's functionality and surroundings. Section 3 (System Features) has the detailed requirement specifications with Sections 4 and 5 talking about the interface requirements and nonfunctional properties such as the performance and reliability, respectively.

## 1.4 Project Scope

Escape Sequence is a survival game with blackjack mechanics. Players are trying to get out of a space ship that is going to crash by completing three levels (rounds) of card challenges. Each turn is harder and new specialty cards with new powers are added.

The game features a single-player mode (vs computer) and a multiplayer mode (up to 6 players). Only three players can win, as there are only three escape pods available.

Providing a fun, logic-based, and interactive gaming experience. Deepening the knowledge of game design, GUI programming, randomization algorithms, and Java-based object-oriented principles. Applying software engineering principles such as modular design, requirement analysis, and documentation.

The final product is in line with the course learning objectives by combining user interaction, algorithmic design, and software development best practices.

## 1.5 References

1. IEEE Std 830-1998, *IEEE Recommended Practice for Software Requirements Specifications*, IEEE Computer Society, 1998.

2. Course Syllabus – CS 491: Software Engineering Project, Dr. Zhao, Western Illinois University, Fall 2025.

3. Team Proposal Document – *Escape Sequence* (The Space Cadets, Sept. 9, 2025).

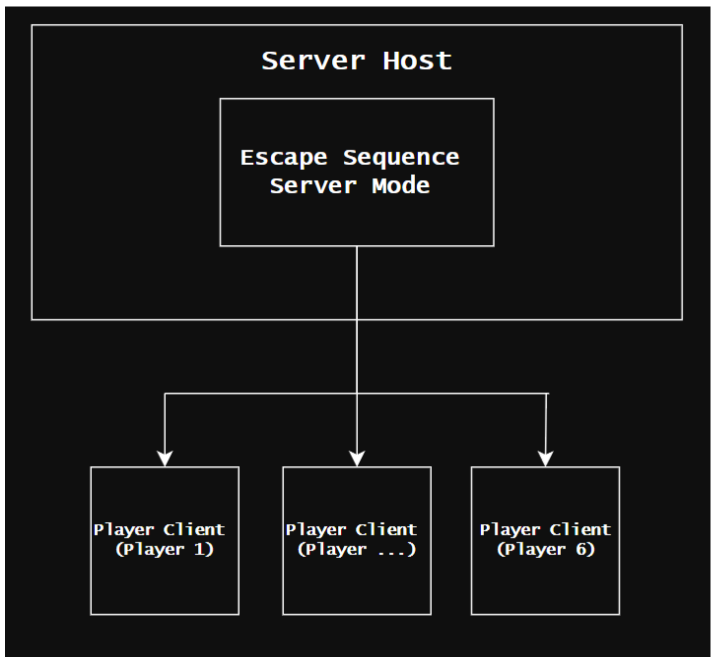
4. Java Documentation – Oracle, *Java SE 17 API Specification*, 2025.

5. Resident Evil 7 DLC “21” and *Mouthwashing* (game design inspirations).

# 2. Overall Description

## 2.1 Product Perspective

Escape Sequence is a standalone multiplayer 2D game developed using NetBeans IDE and Java. The product is self-contained and is not related to any franchises or platforms. It will operate using the Java Virtual Machine (JVM) which allows it to work across platforms. The game will run on most common operating systems with the Java Runtime Environment (JRE) installed. In order to enable multiplayer, the system will have a client-server architecture. There are two external systems that the product will interact with including the host operating system and the player's network.




## 2.2 Product Features

The following is an overview of the major features to be implemented in the game.

- **Title Screen**

  The first screen that pops up once the application is started. It will feature buttons for options starting the game and a written tutorial.

- **Multiplayer Mode**

  After starting the game the host player will select the number of players starting from one going up to 6.

- ## Player Name Entry

  After the host player selects the number of players, the players will enter their name which will be used when referring to them during the game.

- ## View Tutorial

  Due to the more complex rules of the game there will be a short tutorial video before the game starts that can be viewed again at any time as well as a written description of how the game is played displayed in game using a hot key.

- ## Hit

  A core game mechanic that means the player will draw the next card in the deck.

- ## Stay

  Another core game mechanic that means the player will not draw a card this turn.

- ## Use a Specialty Card

  In the second and third round there will be a specialty card dealt to the players that can be played during this round or held to be used in round 3.

- ## Pause Menu

  Will be displayed using a button that is available during gameplay that stops the game for all players with the options of going to the start menu (only for the host player) and viewing the video tutorial.

## 2.3 User Classes and Characteristics

**2.3.1 Primary User Class: Players**

Escape Sequence is heavily inspired by the classic card game Blackjack. While knowledge of Blackjack mechanics would be helpful, it is by no means required to play the game. There are only three in-game moves that the players will make, making the gameplay easy and straightforward. The player's hand total is continuously displayed on screen, eliminating any need for mental math during play. Though Blackjack is generally considered a game of luck, experienced players may apply strategies using statistics and probability when deciding to draw or stay. Players of any skill level should be able to understand the game mechanics. All of the rules and mechanics for the game will be demonstrated in the tutorial video at the start and there will be a written guide available during gameplay.

**User Characteristics:**

- No technical expertise is necessary

- No gaming experience is necessary
- This user class will only be used in single player and multiplayer mode

**2.3.2 Special User Class: Host Player**

In the case of the game being in multiplayer mode, the user to log in and select multiplayer will be the host player. The host player determines the number of players and is the only user that can end the game during gameplay from the pause screen. Outside of these functions the host player has the same user characteristics as regular players.

## 2.4 Operating Environment

The game will be able to run on any standard computer and across several operating systems as long as Java Runtime Environment (JRE) is installed. Windows, MacOS and Linux operating systems will be able to support the game. The software operates as a standalone Java application and relies on the Java Swing Library, which is included with standard JRE installations, for GUI rendering and user interface components. Hardware requirements are minimal and are suitable for most personal computers. Players interact with the game using a keyboard and mouse so both are required to play the game. The players will also need the internet in order to play in multiplayer mode. In the case of multiplayer mode there will be TCP/IP protocols across the clients to synchronize gameplay for all players.

## 2.5 Design and Implementation Constraints

The game is built using Java in the NetBeans IDE utilizing Java Swing will be used for the user interface. Development follows all standard Java programming standards and practices. The application must run seamlessly across Windows , macOS, and major Linux distributions without modification, and must perform adequately on standard consumer hardware. Single player mode must operate entirely offline with no network communication protocols. As an academic project, development is constrained by a two semester timeline. All player data must remain stored locally with input validation to prevent security vulnerabilities. The game must include both video tutorials and written documentation to ensure accessibility for players with varying experience levels.

## 2.6 User Documentation

Escape Sequence provides user documentation through three primary formats accessible directly within the game interface.

1. A video tutorial that demonstrates all core game mechanics including Hit and Stay actions, specialty card effects, round progression, and win/loss conditions. This tutorial is presented before the first game and can be re-accessed at any time through the pause menu.
2. A comprehensive written manual, accessible via hotkey during gameplay, provides detailed explanations of game rules and card effects that players can reference quickly without interrupting the session.

3. Contextual tooltips appear when players hover over specialty cards and game elements, providing just-in-time learning without leaving the current screen.

## 2.7 Assumptions and Dependencies

The successful development and deployment of the application assumes that all players have Java Runtime Environment (JRE) installed, understand basic computer functions, and have hardware meeting the minimum specifications outlined in Section 2.4. The project depends on the continued availability and stability of NetBeans IDE, Java Development Kit (JDK), and standard Java libraries including Java Swing and the Java Collections Framework.

For multiplayer functionality, it is assumed that all players have access to a stable internet connection. The project depends on the availability of Java networking libraries (java.net package) and assumes that players' network configurations, firewalls, and routers allow the necessary TCP/IP communications. Local file systems are assumed to have the appropriate read/write permissions without requiring administrator privileges and reliable file I/O operations to ensure reliable saving and loading of player data.

# 3. System Features

## 3.1 Title Screen

### 3.1.1 Description and Priority

The title screen is the starting point for Escape Sequence and is the first interface players interact with. There are several important options on the title screen including *Single Player, Multiplayer, View Tutorial* and *Exit.* This feature has a high priority **(H)** because it is integral to the game.

### 3.1.2 Stimulus/Response Sequences

1. **Stimulus**: Player launches the game

   **Response**: The title screen is displayed.

2. **Stimulus**: Player clicks *"Single Player"* button.

   **Response**: System starts single player mode and proceeds to the name entry screen.

3. **Stimulus**: Player clicks the *"Multiplayer"* button.

   **Response**: System starts multiplayer mode and current player becomes the host player. Then the game proceeds to the name entry screen.

4. **Stimulus**: Player clicks *"View Tutorial"* button

**Response**: The tutorial video pops up with the written game manual underneath it.

5. **Stimulus**: Player clicks the "*Exit*" button.

   **Response**: A confirmation screen pops-up and if the option is confirmed, the application closes.

### 3.1.3 Functional Requirements

REQ-1: The title screen must appear every time the game is launched.

REQ-2: The title screen shall display the "*Single Player*", "*Multiplayer*", "*View Tutorial*" and "*Exit*" options.

REQ-3: The title screen will appear within 5 seconds of the game being launched.

REQ-4: The options must respond within 0.5 seconds of being clicked.

REQ-5: There shall be a confirmation screen before the player exits the game.

## 3.2 Multiplayer Mode

### 3.2.1 Description and Priority

Multiplayer mode is activated when the player selects the "Multiplayer" option from the title screen. Once the option is clicked the current player becomes the host player and therefore is the only one that can select the number of players and quit the game during game play. When the host player selects the number of players (between 2-6) from the options, the corresponding number of connections for other players to join are opened. There is also an "*Back*" button displayed to go back to the title screen. The multiplayer function is not necessary for the game to run so the priority is low **(L)** for this feature.

### 3.2.2 Stimulus/Response Sequences

1. **Stimulus**: The player clicks on a button indicating the number of players (2-6)

   **Response**: The corresponding number of connections are opened and a code displayed for other players to join

2. **Stimulus**:All other players have joined the game and host player selects "Start"

   **Response**: The game proceeds to the tutorial video

3. **Stimulus**: Player clicks the "*Back*" button.

   **Response**: A confirmation screen pops-up and if the option is confirmed, then it goes back to the title screen.

### 3.2.3 Functional Requirements

REQ-1: The system shall support between 2 and 6 players in multiplayer mode.

REQ-2: The system shall generate a unique 6-character code when a host creates a multiplayer game that is used by the other players to join the game..

REQ-3: The system shall check internet connectivity before allowing multiplayer mode to be accessed, displaying an error message if offline.

REQ-4: The system shall assign the first player as the host player with exclusive privileges to start the game and return to the title screen from pause.

REQ-5: If at any point the host player loses internet connection while in multiplayer mode the game session automatically closes.

REQ-6: If any player disconnects or loses internet connection while in multiplayer mode they are booted out of the game.

## 3.3 Player Name Entry

### 3.3.1 Description and Priority

The player name entry is a function that happens in both single player and multiplayer mode where all players must enter their name in order to start the game. The entry is henceforth used to refer to the player in the game. This is not a necessary function for the game to run and is more of an aesthetic attribute. The priority level for this feature is a low **(L)**.

### 3.3.2 Stimulus/Response Sequences

1. **Stimulus:** In single player mode, the player enters their name in the pop-up

   **Response:** Once a proper name has been entered the start game option will light up.

2. **Stimulus:** The player selects the start game option.

   **Response:** The system proceeds to the tutorial video.

3. **Stimulus**: Player clicks the "*Back*" button.

   **Response**: A confirmation screen pops-up and if the option is confirmed, then it goes back to the title screen.

4. **Stimulus:** In multiplayer mode the host player is prompted to enter their name while the other player joins. The other players must enter their name and the session code to connect to the game.

   **Response:** Once the host player and other players have been connected and logged in that start button will light up.

### 3.3.3 Functional Requirements

REQ-1: The system shall require each player to enter a non-empty name containing letters, numbers, and special characters( !, -, _, ~, *  )

REQ-2: The player's name shall be used when referring to them in game.

REQ-3:  The players names shall stay in the systems memory until the game application is closed

REQ-4:  All players must enter a valid username for that '*Start*' button to light up.

REQ-5: In multiplayer mode, only the host player can select the "*Start*" button.

## 3.4 View Tutorial

### 3.4.1 Description and Priority

View tutorial is a feature seen when the game first starts. It can also be viewed from the title screen or played from the pause menu when selected by the host player. When the video is displayed the written description will accompany it. From the pause menu there will be an option to just view the written instructions. The tutorials are vital for the game to be played by any outside parties, but testing and debugging the system can be done without them. The priority level for this feature is medium**(M)**.

### 3.4.2 Stimulus/Response Sequences

1. **Stimulus:**  Player selects "*View Tutorial"* from the Title Screen or Pause Menu.

   **Response:** The system launches the tutorial video with the written instructions underneath

2. **Stimulus:** The host player clicks the "*Skip Tutorial*" button while the video tutorial is playing at the start of the game.

   **Response:** The tutorial video closes and the first round of the game begins.

3. **Stimulus:** The host player clicks the "*Continue*" button after the video is over

   **Response:** The tutorial video closes and the first round of the game begins.

4. **Stimulus:** The host player clicks the "*Rewatch*" button after the video is over

   **Response:** The tutorial video plays again.

### 3.4.3 Functional Requirements

REQ-1: The system shall play a tutorial video demonstrating gameplay.

REQ-2: The host player or single player shall be allowed to rewatch or exit the tutorial video.

REQ-3: There shall be a written guide accessible via hotkey during gameplay.

## 3.5 Hit

### 3.5.1 Description and Priority

"*Hit*" means the next card in the deck is dealt to the current player. The function is a core game mechanic needed for Escape Sequence to be played therefore the priority level is high **(H)**.

### 3.5.2 Stimulus/Response Sequences

1. **Stimulus:** Player Selects "*Hit*"

   **Response:** System gives the player the next card in the deck and updates their total

2. **Stimulus:** Player selects "*Hit*" but their total is equal to 21 or over

   **Response:** An error message pops up saying they can no longer hit.

### 3.5.3 Functional Requirements

REQ-1: The system shall deal a card from the randomized deck when the player "*Hits*"

REQ-2: After hitting, the player's current hand total is updated within 0.5 seconds.

REQ-3: The system shall not allow a player to hit if their hand total is equal to 21 or above.

## 3.6 Stay

### 3.6.1 Description and Priority

"*Stay*" means no new card is drawn and it is now the next player's turn. The function is a core game mechanic needed for Escape Sequence to be played therefore the priority level is high **(H)**

### 3.6.2 Stimulus/Response Sequences

1. **Stimulus:** Player Selects "*Stay*"

**Response:** The player's turn ends and moves to the next player.

### 3.6.3 Functional Requirements

REQ-1: The game shall move to the next player if the current player selects "*Stay*".

REQ-2: The round shall only conclude when all players stay.

## 3.7 Use Specialty Card

### 3.7.1 Description and Priority

The "*Use Specialty Card"* mechanic allows for special events that benefit the player. The function is only active during the second and third rounds. The specialty cards are (Shield, Wild, Reverse, Freeze, Swap, Peek). Since this mechanic is a core aspect of the second and third rounds of the game the priority is high**(H)**.

### 3.7.2 Stimulus/Response Sequences

1. **Stimulus:** Player selects a specialty card.

   **Response:** The card effect is executed and updates the players hand total or any other players totals that has been effected.

2. **Stimulus:** Player hovers over a specialty card.

   **Response:** A simple text description of the card is displayed.

### 3.7.3 Functional Requirements

REQ-1: The system shall deal one specialty card to each player during rounds two and three.

REQ-2: The specialty card shall carry on to round three if it is not used in round two.

REQ-3: Once the specialty card is selected the correct effect shall be executed.

REQ-4: The available specialty cards are a user shall be displayed only to the player possessing it until it is used.

## 3.8 Pause Menu

### 3.8.1 Description and Priority

The Pause Menu allows the host player or single player to temporarily stop the game, view the tutorials or exit the game. This feature has medium priority, because while it is not integral to the game mechanics, it is still required for game management.

### 3.8.2 Stimulus/Response Sequences

1. **Stimulus:** Host player or single player presses the pause key during gameplay

   **Response:** The game freezes and the pause menu is displayed.

2. **Stimulus:** The "Return to Title Screen" option is selected.

   **Response:** A confirmation screen pops-up and if the option is confirmed, the game closes and returns to the title screen.

3. **Stimulus:** The "View Tutorial" option is selected.

   **Response:** The video tutorial with the written instructions is displayed.

4. **Stimulus**: Player clicks the "*X*" button.

   **Response**: The game resumes.

### 3.8.3 Functional Requirements

REQ-1: The pause button will not be available for players that are not the host in multiplayer mode.

REQ-2: The host player or single player shall be able to return to the title screen from the pause menu.

REQ-3: The game shall resume once the player selects the "*X*" button.

REQ-4: The system shall allow the tutorial to be viewed while the game is paused.

# 4. External Interface Requirements

## 4.1 User Interfaces

The Escape Sequence game features a spaceship-themed graphical user interface (GUI) that not only captivates the players by virtually transferring them to the game but also keeps the interaction simple and clear. The interface is aimed at being user-friendly, attractive, and uniform in terms of design and functionality throughout the game.

First of all, the main menu is a spaceship control panel visually offering four choices; single-player, multiplayer, instructions, and exit.

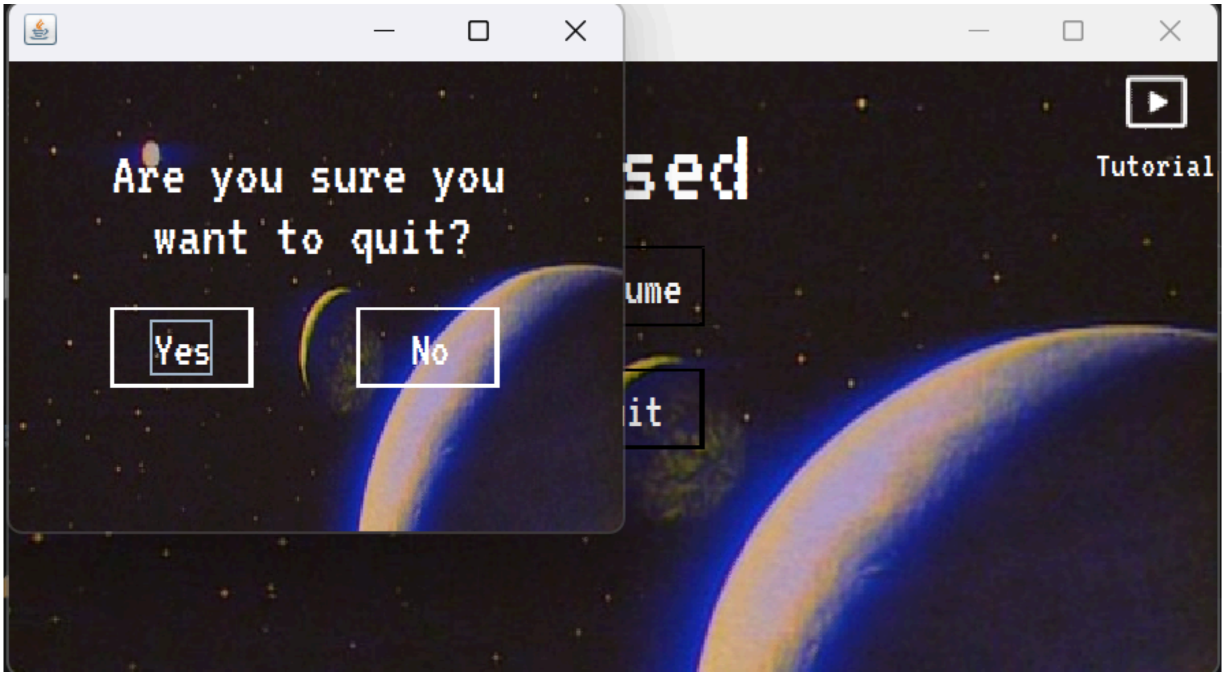
### Title Interface:


The game screen presents the player(s) and computer cards, specialty cards, player scores, and round progress. The format here represents the inside of a spaceship graphically.

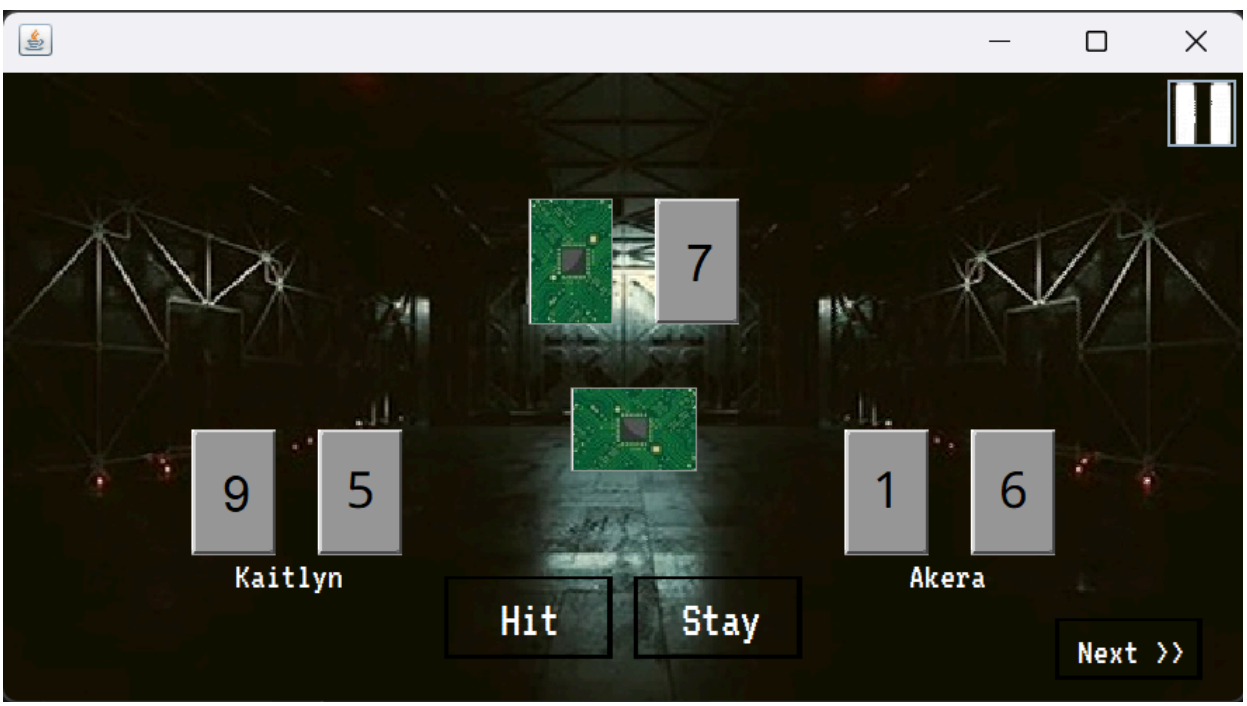
- The Specialty Card Panel is a specific place where the users are shown the available specialty cards (Shield, Wild, Reverse, Freeze, Swap, Peek) with the help of the icons that interact with the users and correspond to the spaceship theme.
- Help, pause, exit, and settings buttons are depicted as spaceship control buttons and are visible on all the screens of the interface.
- Error messages are shown as alert panels that have the style of ship warnings (flashing red indicators) directed at invalid operations.
- Also, players are given an option to use keyboard shortcuts through which they can execute the required operations quickly (H = Hit, S = Stay).
- Sounds and animations the users experience are the same as those of the accompanying spaceship events (alarms). The in-game spaceship manual is the user documentation that informs the users about the game rules and card effects and is reachable from the main menu.
- The description of the User Interface Specification with detailed screen mock-ups, component layouts, and interaction flows will be consistent with the spaceship theme.

## Pause Interface:

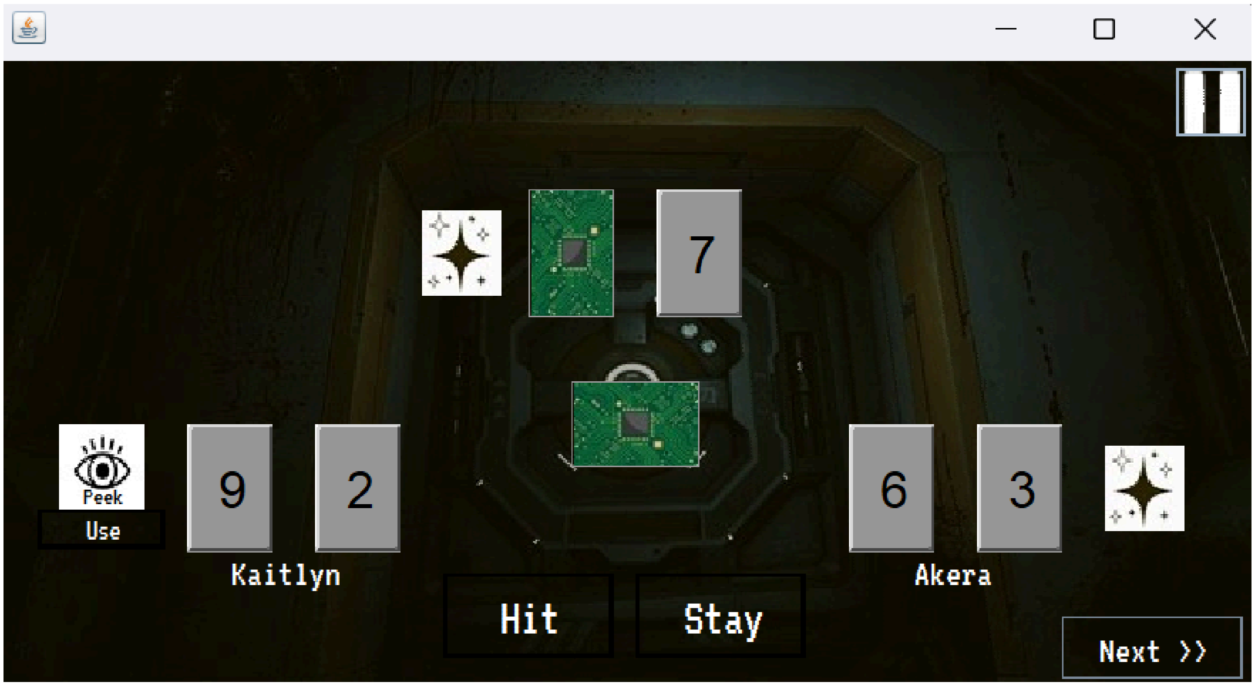
### Pause with Confirmation Interface:



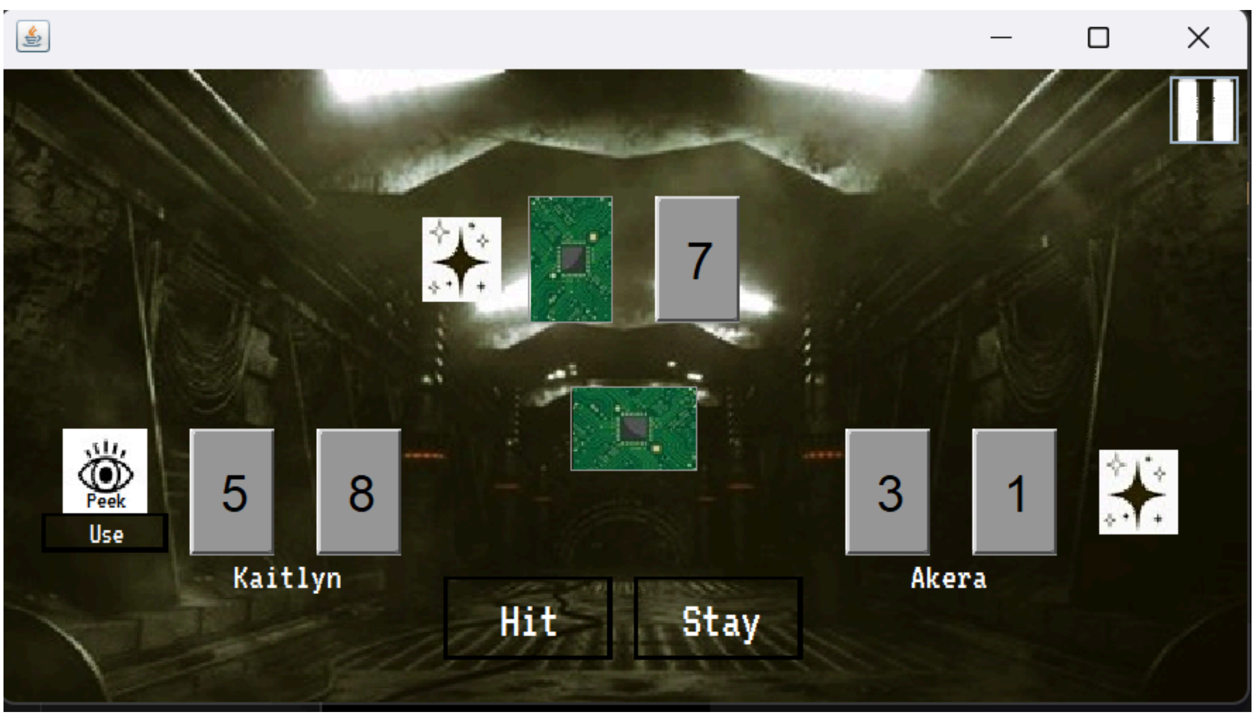

### Round One Interface:



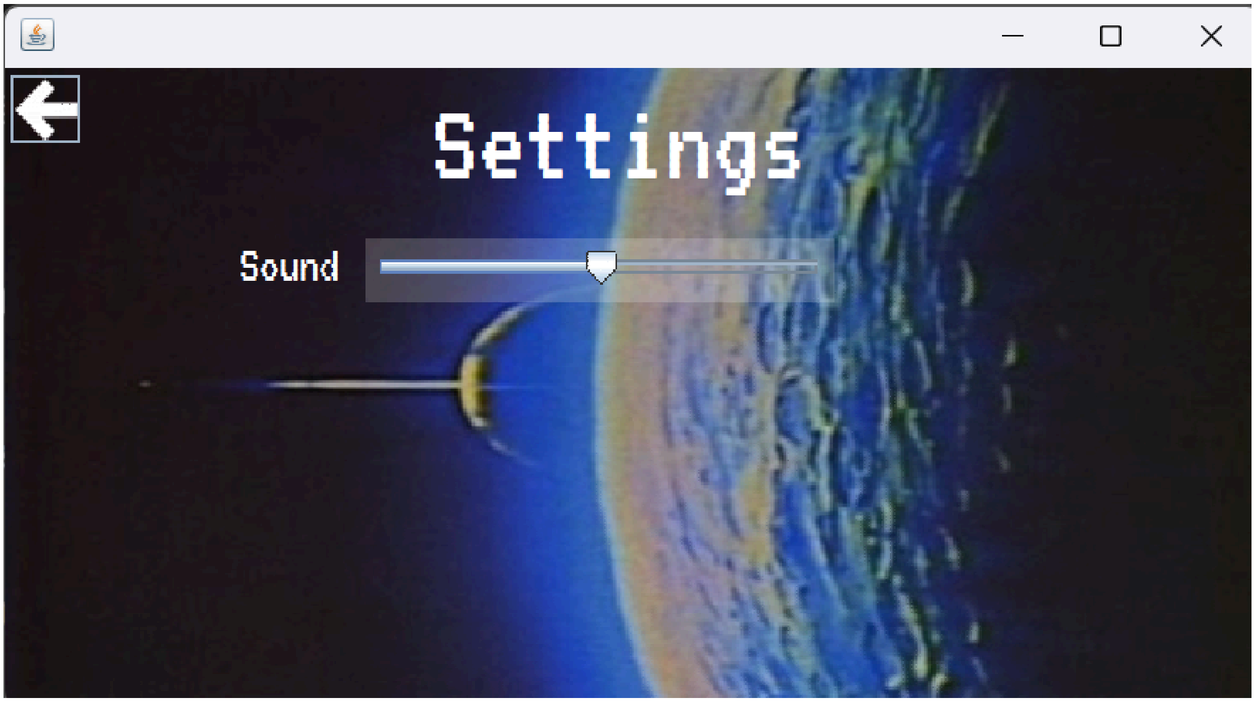
## Round Two Interface:



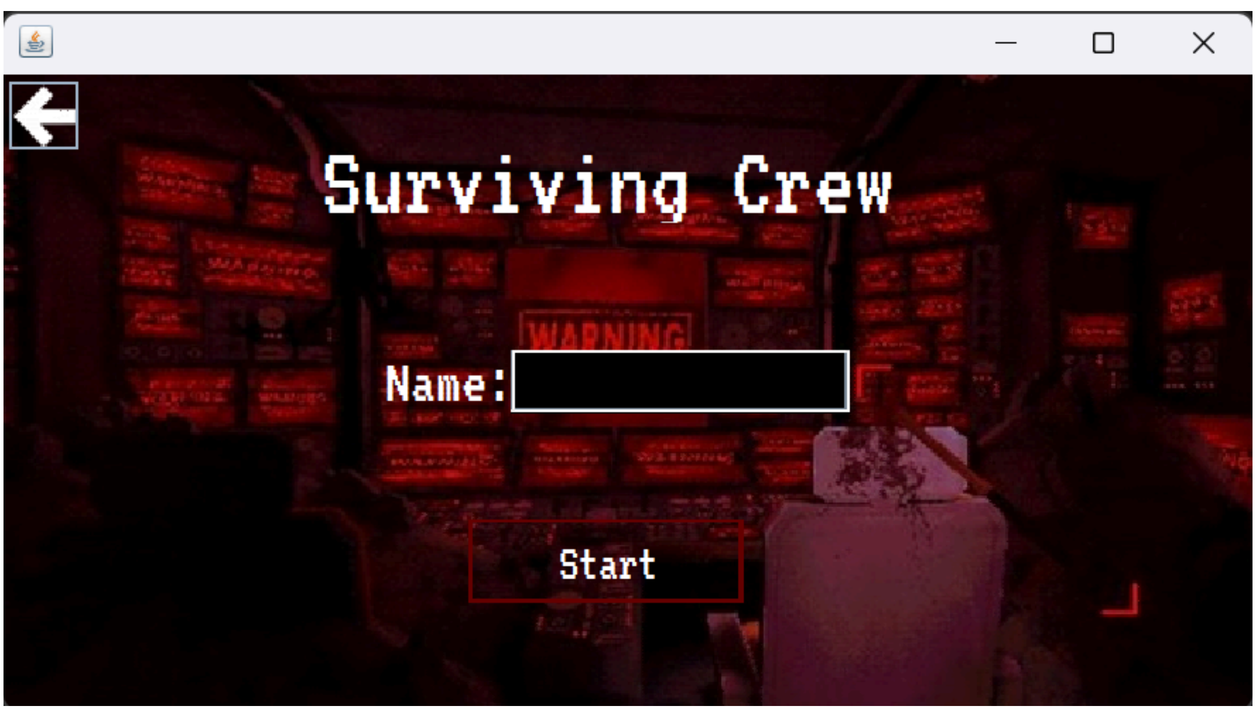

## Round Three Interface:



**Settings Interface:**

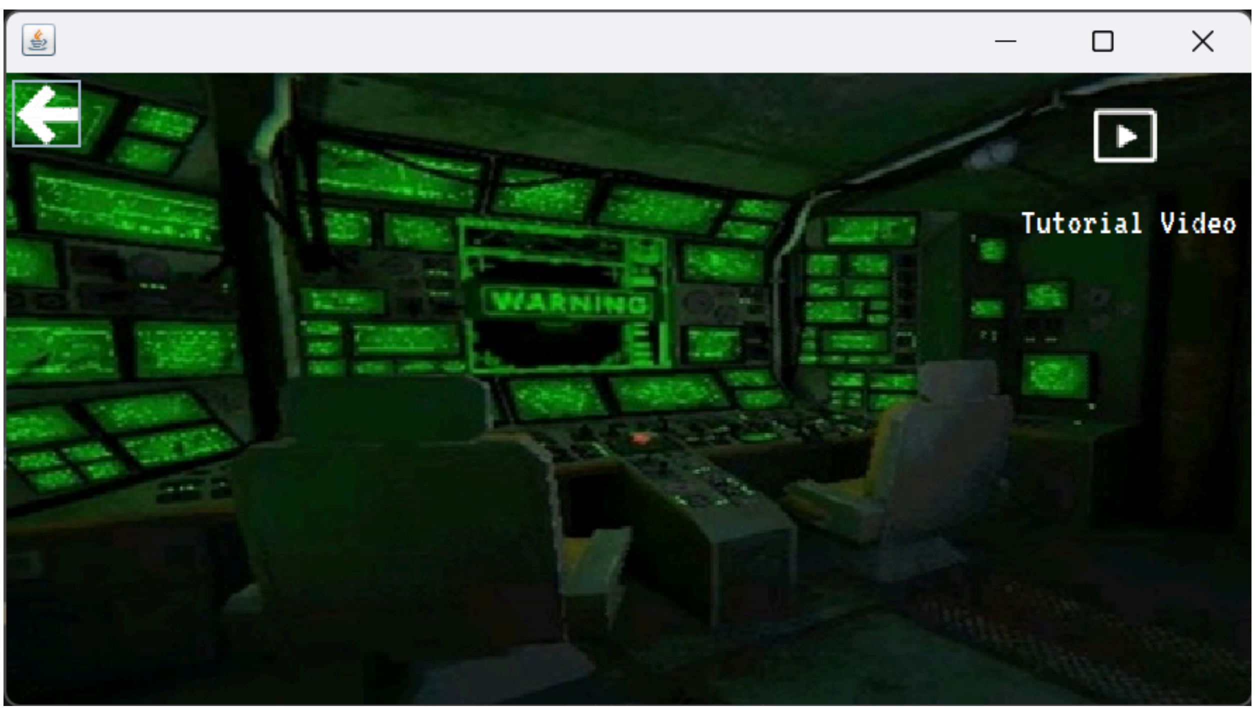



**Single Player Interface:**



**Multiplayer Interface:**


**Rules Interface:**



## 4.2 Hardware Interfaces

Escape Sequence is specifically intended for standard desktop or laptop computers that support the following specifications or capabilities:

- Supported platforms - A desktop or laptop computer running either Windows, macOS, or Linux
- Input - Accepts either a keyboard and mouse combination or available touch input
- Output - Display (monitor) and must provide speakers (for audio cues)
- Hardware: The game will use system resources; CPU, RAM, and graphical capabilities to render 2D graphics and run game logic. Hardware will not be required other than the typical requirements of personal computing devices (or peripherals).

## 4.3 Software Interfaces

Escape Sequence works with numerous software components for functionality:

- Programming Environment: The application will operate in the Java IDE 11 (or higher versions) environment.
- Libraries: To create User Interfaces, Java Swing will be employed; and the Java Collections Framework (JCF) will be used for data structures.
- Operating Systems: The application will run on Windows 10/11, macOS 12+, or Linux Distributions with Java Runtime installed.
- Database/Storage: Player profiles, scores, and saved game states will be stored locally on the file system (JSON or serialized objects) as its persistence method.
- Constraints: Data sharing between components is through modular Java classes and methods to maintain encapsulation and minimize coupling.
- Data Flow:
  - Input Data: Player names, user actions (e.g., hit, stay, use a specialty card), and selections for game configuration.
  - Output Data: Game state, player scores, computer actions, and round results.

## 4.4 Communications Interfaces

Escape Sequence does not necessitate communication outside the local network for any of the core gameplay experience. However, the software may contain optional components or features to the core experience:

- Communication during local multiplayer: In case of several players using one computer, the local GUI will handle the input and output, and internal event handling will be the communication way behind the scenes.
- Protocols: There will be no external communication protocols (FTP, HTTP, or TCP/IP) utilized for version 1.0.
- Security Issues: All the saved data files (player names and score information) are kept locally. Therefore, no sensitive data is going to be sent as external data.

- Changes to be made later: At some point, an online multiplayer feature might be introduced in a subsequent version, thus requiring network protocol specifications and encryption standards for secure data transfer.

# 5. Other Nonfunctional Requirements

## 5.1 Performance Requirements

The Escape Sequence game must function appropriately and consistently under various gameplay situations to allow for a seamless user experience. The following performance requirements are applicable:

- Startup Time: The application shall launch and be interactive within a few seconds on the standard hardware.
- Gameplay Responsiveness: Players' actions (card draws, specialty cards, menu navigation) shall respond with a latency of less than one (1) second.
- Frame Rate: The GUI shall achieve a minimum of 30 frames per second (FPS) including animations and visual effects to produce a steady transition between gameplay elements.
- Multiplayer Mode Synchronization: In multiplayer mode, all players' states and actions must be synchronized between their respective game instances within two (2) seconds of the turn event.
- Memory Usage: The game must not use more than 500 MB of RAM during runtime to maintain stable performance on multiple systems.
- Error Handling and Recovery: The system must recover from minor runtime errors (missing files, invalid input) with a graceful error with minimal recovery time that does not require a full system restart.
- Database/Storage Access: Player data (scores, names, results) must be written to and read back from the local database in information.

## 5.2 Safety Requirements

While Escape Sequence is a digital game, some safety requirements must still be satisfied to protect the user's data, and ensure that the user engages with the product in a responsible manner.

- Data Integrity Protection: The system must protect against data loss or corruption during reading and/or writing data to disk through the use of appropriate file system handling, validation, and exception handling.
- Error Prevention: The program will safeguard against unintended consequences of unsafe operations, such as overwriting an essential file, deleting player stored data without notifying the player, or confirming the accessibility of system directories with restricted permissions.

- Crash Prevention: The application will quickly handle unexpected error conditions (missing resources, null reference) through controlled exception handling that will not result in any application crashes.
- 
- User Input Validation: All player inputs will be validated prior to use to avoid injection of harmful data or disruption of the local database.
- System Resource Safety: The software will not perform operations that could adversely affect the host system (excessive CPU usage, infinite looping conditions, or unsafe memory allocation).
- Compliance and Ethics: The game's software must conform to safe practices in software development and data protection practices. In the event that online play is integrated into the game in a future version, networks will also have to conform to safe practices (use of security, encryption).

## 5.3 Security Requirements

The Escape Sequence platform must properly utilize player data including usernames, gameplay outcomes, and scores and be protected from any unauthorized attempt. The following measures will be established to ensure the following protections are in place:

- Authenticator: During the game, each player will be required to enter a unique username. In the event that the game requires an online platform, then passwords or other authenticators could be added to ensure unique authentication.
- Player Data Protection: Players data (username, scores, success/failure) will be secured either in a database or protected file format. Even if the application was interrupted during play, player data should successfully be validated, stored, and kept secure.
- Administrators: Only authenticated players (game players) would interact with the system. Administrator or developer access to system data could be kept separate by creating another configuration file that would remain inaccessible through the game user interface.
- Inputs: User inputs when playing the game, including player names and in-game selections will be input validated to protect against injection attacks and data corruption.
- Protected Data: No personal data will be also collected nor shared beyond the limited amount required for gameplay. To ensure player privacy, all stored data for the game will remain on local player systems except to a player who clearly states that he/she accepts/consents to and to share the protected data.

## 5.4 Software Quality Attributes

Escape Sequence software aims to deliver a top-notch, dependable, and enjoyable experience for its users. These quality attributes for software serve as the primary goals for the development and assessment of the Escape Sequence.

- Usability. The GUI will be attractive and intuitive for the users and, hence, new users of the game will be able to understand it by themselves, thus, instruction time will be minimal.
- Reliability. The system will be capable of detecting inappropriate input, thus, a minor fault recovery (such as a lost multiplayer connection or an invalid input) will be possible without the system shutting down.
- Performance. The playing period should be a few seconds, and users should not notice any lag during the game on standard equipment.
- Maintainability. The code base will be modular, which implies that the developers will have made separate classes for players, cards, and game logic, each class being at least somewhat self-contained. The commenting and documentation will be done according to Java standards so that the process of debugging and updating later will be easier.
- Portability. Escape Sequence being a Java program, an application running on any OS with Java Runtime Environment (Windows, macOS, Linux) should be able to run the Escape Sequence.
- Robustness. The system will be equipped with error-handling routines designed for unexpected input, missing resources, or incomplete data files that will avert program crashes.

# 6. Other Requirements

## Appendix A: Glossary

**Software Requirements Specification (SRS) -** a formal document that defines the purpose, scope, and detailed requirements of a software system, including its functional and nonfunctional specifications.

**Functional -** relating to the specific actions, operations, or tasks that a software system performs to meet its objectives.

**Nonfunctional -** relating to the quality attributes of a software system, such as performance, reliability, usability, and security, rather than the specific behaviors it performs.

**Java-based -** developed or implemented using the Java programming language and its associated libraries or frameworks.

**Interactive -** designed to allow active user participation and real-time feedback between the user and the system.

**2D -** referring to two-dimensional visual elements or graphics displayed on a flat plane without depth perception.

**First-person -** relating to a game or simulation perspective in which the player experiences the environment through the eyes of the character.

**Functional behavior -** the observable actions and responses of a software system that fulfill its defined functional requirements.

**External interfaces -** points of interaction between a software system and other systems, hardware, or users through which data or control signals are exchanged.

**Constraints -** limitations or conditions imposed on a system's design, development, or operation, such as hardware capabilities, regulations, or performance requirements.

**Single-player -** designed for play by one user interacting with the computer or game environment.

**Multiplayer modes -** configurations of a game or system that allow two or more players to participate simultaneously, either cooperatively or competitively.

**Graphical user interface (GUI)** - a visual interface that allows users to interact with software through graphical elements such as buttons, icons, and menus instead of text commands.

**Subsystems -** smaller, independent components within a larger software system that perform specific functions and interact with other components to achieve overall system goals.

**Algorithm** - a defined sequence of logical steps or instructions used by a computer program to perform a calculation, solve a problem, or process data.

**Database -** an organized collection of data stored electronically and managed by software to allow efficient retrieval, modification, and storage.

**Randomizer -** a function or algorithm that produces random or pseudorandom outcomes, used to shuffle cards or determine unpredictable results in a game.

**Specialty card -** a unique card with special effects or abilities that alter normal gameplay, such as reversing a move or protecting a player.

**Keycard -** a digital or virtual card used within the game to unlock new levels or allow progression; awarded to players who win a round.

**Dealer** - the computer-controlled opponent in the game that distributes cards and plays against the user or other players.

**Scoreboard -** a digital display or record that tracks player performance, including wins, losses, and survival outcomes.

**Game round -** a single stage of gameplay in which players draw cards, make decisions, and attempt to achieve a winning hand.

**Player** - an individual participant in the game, either a human user or a computer-controlled entity.

**Escape pod -** a virtual objective within the game representing survival or victory; only available to players who collect the required keycards.

**System failure -** a simulated event within the game narrative indicating malfunction or breakdown of the virtual spaceship's systems.

**Shield -** a type of specialty card that allows a player to discard their last drawn card without penalty.

**Requirement -** a condition or capability that a software system must possess or a constraint it must satisfy to meet user or stakeholder needs.

**Use case -** a written description of how a user interacts with a system to achieve a specific goal or complete a task.

**Module -** a self-contained unit of code that performs a specific function and can be integrated with other modules to form a complete system.

**API (Application Programming Interface) -** a set of defined methods, protocols, and tools that allow different software components or systems to communicate and share data.

**Testing -** the process of evaluating a software system or component to verify that it meets specified requirements and performs as intended.

**Bug -** an error, flaw, or fault in a software program that causes incorrect or unexpected behavior.

**Iteration -** a single cycle in an incremental development process where the software is refined, tested, and improved.

**Prototype -** an early version of a software system created to test concepts, interfaces, or functionality before full-scale development.

**Performance -** the measure of how efficiently a software system executes tasks, often evaluated in terms of speed, responsiveness, and resource usage.

**Reliability -** the ability of a software system to operate without failure under specified conditions for a defined period.

**Usability -** the degree to which a software system is easy to learn, understand, and operate by its intended users.

**Security** - the protection of software and data against unauthorized access, alteration, or destruction.

**Data structure -** a specialized format for organizing, processing, and storing data efficiently in a computer program.

**Input validation -** the process of ensuring that user-provided data is correct, safe, and within acceptable limits before being processed by the system.

# Appendix B: Analysis Models

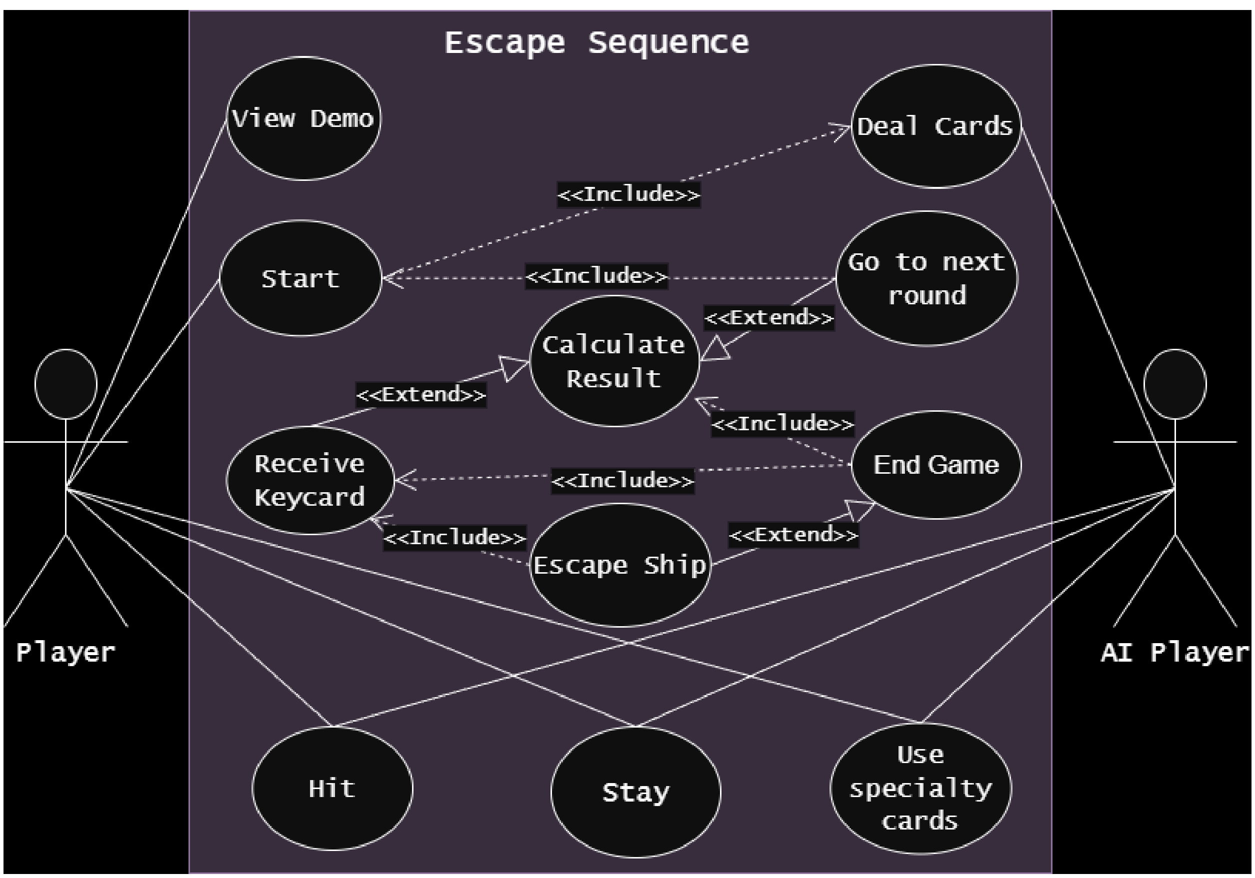


*Figure 6.1: Use Case Diagram*

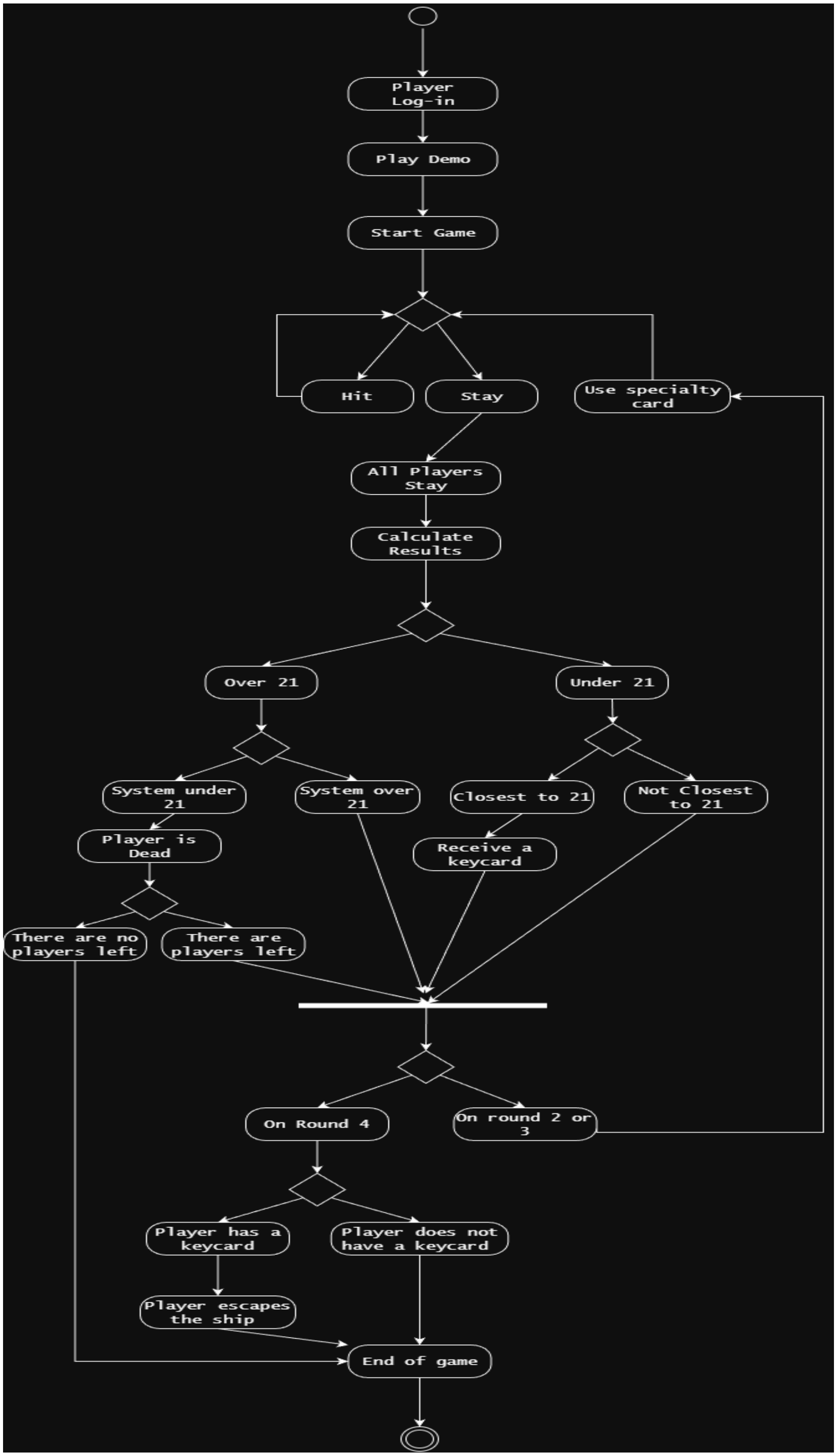



*Figure 6.2: Activity Diagram*

# Appendix C: Issues List

| Issue ID | Description | Status | Assigned To | Notes / Next Steps |
|---|---|---|---|---|
| **ISS-01** | Determination of how multiplayer will be implemented (local or online). | Pending | TBD | Consider evaluating the feasibility of socket programming and/or local turn-based play before settling on final design decisions. |
| **ISS-02** | Balancing of specialty card mechanics (such as card effects of “wild” or “swap”). | In Progress | TBD | Testing should be done to see when and where these attributes need to be used. |
| **ISS-03** | Design of the database for player data and a scoreboard. | Pending | TBD | Determine the best type of database system to be used. |
| **ISS-04** | Specifics on the GUI layout and final assets for the spaceship theme visuals. | In Progress | TBD | Discuss and design full mock-ups of the interface. |